

COURSEWORK & STUDY GUIDE

JavaScript & TypeScript

Every concept explained with code — from JS fundamentals to advanced type-level programming

Phase 1 through Phase 6 · Complete Reference

PHASE 1

JavaScript Fundamentals

1.1 Variables & Scoping

JavaScript has three ways to declare variables. Understanding which to use — and why — is the first step to writing predictable code.

var — the old way (avoid in modern code)

var is function-scoped, not block-scoped. It also gets 'hoisted' to the top of its function, which causes surprising bugs.

```
// var is FUNCTION scoped
function example() {
  if (true) {
    var x = 10; // declared inside block...
  }
  console.log(x); // ...but accessible here! Outputs: 10
}

// var hoisting - declaration moves to top, but not value
console.log(name); // undefined (not an error!)
var name = 'Alice';
console.log(name); // 'Alice'
```

let — block-scoped, reassignable

```
let count = 0;
count = 1; // OK - reassignment allowed

if (true) {
  let blockVar = 'only here';
}
// console.log(blockVar); // ReferenceError - not accessible outside block

// Temporal Dead Zone (TDZ) - cannot use before declaration
// console.log(age); // ReferenceError
let age = 25;
```

const — block-scoped, not reassignable

```
const PI = 3.14159;
```

```
// PI = 3; // TypeError: Assignment to constant variable

// IMPORTANT: const on objects only prevents reassignment of the binding.
// The object itself can still be mutated.
const user = { name: 'Alice' };
user.name = 'Bob'; // OK - mutating the object is allowed
// user = {};      // TypeError - cannot reassign the variable
```

Rule of Thumb

Always use `const` by default. Use `let` only when you know the variable will be reassigned. Never use `var` in modern code.

1.2 Data Types & Type Coercion

JavaScript has 7 primitive types and one complex type (Object). Understanding them — and how JS silently converts between them — is critical.

The 7 Primitive Types

```
let str    = 'hello';           // string
let num    = 42;                // number (no int vs float - all are number)
let float  = 3.14;              // also number
let bool   = true;              // boolean
let nothing = null;             // null (intentional absence of value)
let missing = undefined;        // undefined (unintentionally absent)
let sym     = Symbol('id');      // symbol (unique identifier)
let bigInt  = 9007199254740993n; // bigint (very large integers)
```

Type Coercion — JS's Silent Conversions

JavaScript automatically converts types in many situations. This is one of JS's most error-prone features — and exactly why TypeScript exists.

```
// Addition with a string: coerces number to string (concatenation)
console.log('5' + 1); // '51' - number becomes string
console.log('5' + true); // '5true'

// Other math operators: coerce string to number
console.log('5' - 1); // 4
console.log('5' * 2); // 10
console.log('5' - true); // 4 (true becomes 1)

// Equality coercion: == vs ===
console.log(5 == '5'); // true - loose equality (coerces types)
console.log(5 === '5'); // false - strict equality (no coercion)
```

```
console.log(null == undefined); // true
console.log(null === undefined); // false

// Falsy values (coerce to false in boolean context)
// false, 0, '', null, undefined, NaN
if (0) { /* never runs */ }
if ('') { /* never runs */ }
if (null) { /* never runs */ }
```

Why This Matters for TypeScript

TypeScript's type system exists specifically to prevent implicit coercion bugs. When you declare a variable as string, TS won't let you accidentally add a number to it.

1.3 Functions

Functions are first-class citizens in JavaScript — they can be assigned to variables, passed as arguments, and returned from other functions.

Function Declarations vs Expressions

```
// Function Declaration - hoisted, callable before definition
function greet(name) {
  return 'Hello, ' + name;
}

// Function Expression - not hoisted
const greet2 = function(name) {
  return 'Hello, ' + name;
};

// Arrow Function - shorter syntax, no own 'this'
const greet3 = (name) => 'Hello, ' + name;

// Multi-line arrow function
const greet4 = (name) => {
  const message = 'Hello, ' + name;
  return message;
};
```

Default Parameters & Rest/Spread

```
// Default parameters
function createUser(name, role = 'viewer', active = true) {
  return { name, role, active };
}
createUser('Alice'); // { name: 'Alice', role: 'viewer', active: true }
```

```
createUser('Bob', 'admin'); // { name: 'Bob', role: 'admin', active: true }

// Rest parameters - collect remaining args into an array
function sum(...numbers) {
  return numbers.reduce((total, n) => total + n, 0);
}
sum(1, 2, 3, 4); // 10

// Spread operator - expand array into arguments
const nums = [1, 2, 3];
Math.max(...nums); // 3
```

Higher-Order Functions

```
// Functions that take or return other functions
function multiply(factor) {
  return (number) => number * factor; // returns a function
}

const double = multiply(2);
const triple = multiply(3);

double(5); // 10
triple(5); // 15

// Common array higher-order functions
const numbers = [1, 2, 3, 4, 5];

numbers.map(n => n * 2); // [2, 4, 6, 8, 10]
numbers.filter(n => n % 2 === 0); // [2, 4]
numbers.reduce((acc, n) => acc + n, 0); // 15
```

1.4 Objects & Arrays

Objects

```
// Object literal
const user = {
  name: 'Alice',
  age: 30,
  greet() { // method shorthand
    return `Hi, I'm ${this.name}`;
  }
};

// Property access
user.name; // dot notation
user['age']; // bracket notation (useful for dynamic keys)

// Destructuring
```

```
const { name, age } = user;
const { name: userName } = user; // rename on destructure

// Spread operator – shallow copy or merge
const updated = { ...user, age: 31 };

// Optional chaining – safe property access
const street = user?.address?.street; // undefined (no error)

// Nullish coalescing – fallback for null/undefined
const city = user?.address?.city ?? 'Unknown';
```

Arrays

```
const fruits = ['apple', 'banana', 'cherry'];

// Common methods
fruits.push('date');           // add to end
fruits.pop();                  // remove from end
fruits.unshift('avocado');     // add to beginning
fruits.shift();                // remove from beginning

// Finding items
fruits.indexOf('banana');      // 1
fruits.find(f => f.length > 5); // 'banana'
fruits.includes('cherry');     // true

// Array destructuring
const [first, second, ...rest] = fruits;

// Spread to copy
const copy = [...fruits];
const combined = [...fruits, 'elderberry'];
```

1.5 Prototypes & the 'this' Keyword

Before ES6 classes, JavaScript used prototypes for inheritance. Understanding this is key to understanding how JS classes actually work under the hood.

Prototype Chain

```
// Every object has a hidden [[Prototype]] link
const animal = {
  speak() { return `${this.name} makes a sound.`; }
};

const dog = Object.create(animal); // dog's prototype = animal
dog.name = 'Rex';
```

```
dog.speak(); // 'Rex makes a sound.' — found on prototype

// Constructor functions (old-style classes)
function Person(name, age) {
  this.name = name;
  this.age = age;
}
Person.prototype.greet = function() {
  return `Hi, I'm ${this.name}`;
};

const alice = new Person('Alice', 30);
alice.greet(); // 'Hi, I'm Alice'
```

'this' — Context-Dependent

```
const obj = {
  name: 'Alice',
  greet: function() { return this.name; }, // 'Alice'
  greetArrow: () => this.name,           // undefined (no own 'this')
};

// Losing 'this' context
const fn = obj.greet;
fn(); // undefined — 'this' is now the global object (or undefined in strict mode)

// Fix with bind
const bound = obj.greet.bind(obj);
bound(); // 'Alice'

// Fix with arrow function in class
class Counter {
  count = 0;
  increment = () => { this.count++; }; // arrow: 'this' is always the instance
}
```

1.6 Closures

A closure is a function that remembers the variables from its outer (enclosing) scope, even after that outer function has finished executing. This is one of JavaScript's most powerful features.

```
// Basic closure
function makeCounter() {
  let count = 0; // private variable
  return {
    increment: () => ++count,
    decrement: () => --count,
    value: () => count,
  };
};
```

```
}

const counter = makeCounter();
counter.increment(); // 1
counter.increment(); // 2
counter.decrement(); // 1
counter.value();      // 1
// 'count' is private — only accessible via the returned methods

// Practical use: function factories
function createMultiplier(factor) {
  return (n) => n * factor; // 'factor' is captured in closure
}

const triple = createMultiplier(3);
triple(7); // 21

// Closures in callbacks — a classic gotcha
for (let i = 0; i < 3; i++) {
  setTimeout(() => console.log(i), 0); // 0, 1, 2 (correct with let)
}
// With var instead of let: would print 3, 3, 3
```

1.7 Asynchronous JavaScript

JavaScript is single-threaded but non-blocking. The event loop allows it to handle async operations (network, timers, files) without freezing. The async model evolved through three stages.

Stage 1 — Callbacks (Old Way)

```
// Callbacks: pass a function to be called when the async work is done
function fetchUser(id, callback) {
  setTimeout(() => {
    const user = { id, name: 'Alice' };
    callback(null, user); // null = no error
  }, 1000);
}

fetchUser(1, (error, user) => {
  if (error) { console.error(error); return; }
  console.log(user); // { id: 1, name: 'Alice' }
});

// Callback hell — deeply nested, hard to read
getUser(id, (err, user) => {
  getPosts(user.id, (err, posts) => {
    getComments(posts[0].id, (err, comments) => {
      // ... pyramid of doom
    });
  });
});
```



```
});
```

Stage 2 — Promises

```
// Promise: an object representing an eventual value
function fetchUser(id) {
  return new Promise((resolve, reject) => {
    setTimeout(() => {
      if (id > 0) resolve({ id, name: 'Alice' });
      else       reject(new Error('Invalid ID'));
    }, 1000);
  });
}

// Chaining promises - no more pyramid
fetchUser(1)
  .then(user    => getPosts(user.id))
  .then(posts   => getComments(posts[0].id))
  .then(comments => console.log(comments))
  .catch(error  => console.error(error));
```

Stage 3 — async/await (Modern Standard)

```
// async/await: syntactic sugar over Promises - reads like synchronous code
async function loadUserData(id) {
  try {
    const user    = await fetchUser(id);      // waits for promise
    const posts   = await getPosts(user.id);
    const comments = await getComments(posts[0].id);
    return comments;
  } catch (error) {
    console.error('Failed:', error.message);
    throw error; // re-throw if needed
  }
}

// Parallel execution (don't await one by one if independent)
async function loadParallel() {
  const [users, products] = await Promise.all([
    fetchUsers(),
    fetchProducts(),
  ]);
  return { users, products };
}
```

1.8 Error Handling

```
// Basic try/catch/finally
function divide(a, b) {
```

```
    if (b === 0) throw new Error('Division by zero');
    return a / b;
}

try {
  const result = divide(10, 0);
} catch (error) {
  console.error(error.message); // 'Division by zero'
} finally {
  console.log('Always runs - cleanup here');
}

// Custom Error types
class ValidationError extends Error {
  constructor(field, message) {
    super(message);
    this.name = 'ValidationError';
    this.field = field;
  }
}

try {
  throw new ValidationError('email', 'Invalid email format');
} catch (error) {
  if (error instanceof ValidationError) {
    console.log(`Field: ${error.field}`); // 'email'
  }
}
```

PHASE 2

TypeScript Fundamentals

2.1 Type Annotations & Inference

TypeScript's core feature is its type system. You can explicitly tell it what type something is, or let it figure it out automatically.

Type Annotations — Explicit

```
// Variable annotations
let name: string = 'Alice';
let age: number = 30;
let active: boolean = true;

// Function annotations
function add(a: number, b: number): number {
  return a + b;
}

// TypeScript now catches mistakes at compile time
// add('hello', 2); // Error: Argument of type 'string' is not assignable to
// parameter of type 'number'
```

Type Inference — TypeScript Figures It Out

```
// You don't always need to annotate - TS infers from the value
let name = 'Alice'; // inferred as: string
let count = 0; // inferred as: number
let flag = true; // inferred as: boolean

// Inference works through operations too
let doubled = count * 2; // inferred as: number
let greeting = 'Hi ' + name; // inferred as: string

// Function return type inferred
function add(a: number, b: number) {
  return a + b; // return type inferred as: number
}

// Best practice: annotate parameters, let TS infer return types
```

Rule

Let TypeScript infer types where it can. Add explicit annotations for function parameters, public APIs, and anywhere the inference isn't clear.

2.2 Primitive Types

Basic Types

```
let s:    string  = 'hello';
let n:    number  = 42;
let b:    boolean = true;

// null and undefined – only assignable to themselves (with strictNullChecks)
let empty: null      = null;
let missing: undefined = undefined;

// void – function that returns nothing
function log(msg: string): void {
  console.log(msg); // no return value
}
```

Special Types: any, unknown, never

```
// any – escapes the type system (avoid if possible)
let x: any = 'hello';
x = 42;      // OK
x = true;    // OK
x.anything(); // OK – no type checking at all

// unknown – type-safe alternative to any
let value: unknown = 'hello';
// value.toUpperCase(); // Error – cannot use without narrowing
if (typeof value === 'string') {
  value.toUpperCase(); // OK – narrowed to string
}

// never – a type that can never have a value
// Use in functions that always throw or have infinite loops
function fail(message: string): never {
  throw new Error(message);
}

// never is also the result of impossible narrowing
function assertUnreachable(x: never): never {
  throw new Error('Should never reach here');
}
```

unknown vs any

Use unknown instead of any when you don't know the type but still want safety. unknown forces you to check the type before using it. any bypasses all checks.

2.3 Arrays & Tuples

```
// Array types - two syntaxes, both equivalent
let nums1: number[] = [1, 2, 3];
let nums2: Array<number> = [1, 2, 3];

let strs: string[] = ['a', 'b', 'c'];
let flags: boolean[] = [true, false, true];

// Array of objects
interface User { name: string; age: number; }
let users: User[] = [{ name: 'Alice', age: 30 }];

// Readonly arrays - cannot be mutated
const frozen: readonly number[] = [1, 2, 3];
// frozen.push(4); // Error

// Tuples - fixed length, fixed types at each position
let point: [number, number] = [10, 20];
let entry: [string, number] = ['Alice', 30];

// Named tuple elements (TS 4.0+)
let coord: [x: number, y: number, z: number] = [1, 2, 3];

// Destructuring tuples
const [x, y] = point;

// Rest elements in tuples
type StringAndNumbers = [string, ...number[]];
const data: StringAndNumbers = ['header', 1, 2, 3, 4];
```

2.4 Enums

Enums let you define a set of named constants. They make code more readable when a variable can only be one of a fixed set of values.

```
// Numeric enum - auto-increments from 0
enum Direction {
  Up, // 0
  Down, // 1
  Left, // 2
  Right, // 3
}
let move: Direction = Direction.Up;

// String enum - more readable, recommended for most cases
enum Status {
  Active = 'ACTIVE',
  Inactive = 'INACTIVE',
}
```

```
    Pending = 'PENDING',
  }
let userStatus: Status = Status.Active;
console.log(userStatus); // 'ACTIVE'

// Const enum — inlined at compile time, no runtime object
const enum Size { Small, Medium, Large }
let s: Size = Size.Medium; // compiles to: let s = 1

// Using enums in functions
function getLabel(status: Status): string {
  switch (status) {
    case Status.Active:    return 'Active User';
    case Status.Inactive:  return 'Inactive User';
    case Status.Pending:   return 'Pending Approval';
  }
}
```

2.5 Union & Intersection Types

Union Types — OR

```
// A value that can be one of several types
let id: string | number;
id = 'abc123'; // OK
id = 42;       // OK
// id = true;  // Error

// Union in function parameters
function formatId(id: string | number): string {
  // Must handle both types — narrowing is required
  if (typeof id === 'number') {
    return id.toString().padStart(6, '0');
  }
  return id.toUpperCase();
}

// Union with interfaces
interface Cat { meow(): void; }
interface Dog { bark(): void; }
type Pet = Cat | Dog;
```

Intersection Types — AND

```
// Combine two types — must satisfy both
interface HasName { name: string; }
interface HasEmail { email: string; }

type Contact = HasName & HasEmail;

const c: Contact = {
```

```
    name: 'Alice', // required (from HasName)
    email: 'alice@example.com', // required (from HasEmail)
  };

// Practical: extending types with mixins
type WithTimestamps<T> = T & {
  createdAt: Date;
  updatedAt: Date;
};

interface User { name: string; role: string; }
type UserRecord = WithTimestamps<User>;
// Must have: name, role, createdAt, updatedAt
```

2.6 Interfaces

Interfaces define the shape of an object. They are the most common way to describe structured data in TypeScript.

```
// Basic interface
interface User {
  id: number;
  name: string;
  email: string;
  age?: number; // optional property
  readonly createdAt: Date; // cannot be changed after creation
}

// Using the interface
function createUser(data: User): User {
  return data;
}

// Interface with methods
interface Repository<T> {
  findById(id: number): T | null;
  findAll(): T[];
  save(entity: T): T;
  delete(id: number): void;
}

// Extending interfaces
interface Animal { name: string; }
interface Pet extends Animal { owner: string; }
interface ServicePet extends Pet { certification: string; }

// Index signatures – objects with dynamic keys
interface StringMap {
  [key: string]: string; // any string key maps to string value
}
const labels: StringMap = { hello: 'world', foo: 'bar' };
```

Interface vs Type Alias

```
// Both can describe object shapes
interface UserI { name: string; age: number; }
type UserT = { name: string; age: number; };

// Key difference: interfaces can be merged (declaration merging)
interface Window { myApp: string; } // extends built-in Window
interface Window { version: number; } // merges - Window now has both

// Type aliases can describe unions and primitives - interfaces cannot
type ID = string | number; // OK with type alias
// interface ID = string | number; // Error - not valid
```


PHASE 3

Generics & Advanced Types

3.1 Generics

Generics allow you to write functions and types that work with any type while still maintaining full type safety. They are the most powerful feature in TypeScript.

The Problem Generics Solve

```
// Without generics: must choose between type safety and reusability
function getFirstAny(arr: any[]): any { return arr[0]; }
const first = getFirstAny([1, 2, 3]);
first.toFixed(2); // No error but could crash if arr were strings

// With generics: type-safe AND reusable
function getFirst<T>(arr: T[]): T { return arr[0]; }

const n = getFirst([1, 2, 3]); // T inferred as: number
const s = getFirst(['a', 'b', 'c']); // T inferred as: string

n.toFixed(2); // OK - TS knows it's a number
s.toUpperCase(); // OK - TS knows it's a string
```

Generic Functions

```
// Multiple type parameters
function pair<A, B>(first: A, second: B): [A, B] {
  return [first, second];
}
const p = pair('hello', 42); // [string, number]

// Generic with constraints - T must have a .length property
function getLength<T extends { length: number }>(item: T): number {
  return item.length;
}
getLength('hello'); // 5 - string has .length
getLength([1, 2, 3]); // 3 - array has .length
// getLength(42); // Error - number has no .length

// Constraint: T must be a key of U
function getProperty<T, K extends keyof T>(obj: T, key: K): T[K] {
  return obj[key];
}
const user = { name: 'Alice', age: 30 };
getProperty(user, 'name'); // string
getProperty(user, 'age'); // number
```

```
// getProperty(user, 'email'); // Error - 'email' not in User
```

Generic Interfaces & Classes

```
// Generic interface
interface ApiResponse<T> {
  data: T;
  status: number;
  message: string;
}

type UserResponse = ApiResponse<User>;
type ProductResponse = ApiResponse<Product[]>;

// Generic class
class Stack<T> {
  private items: T[] = [];

  push(item: T): void { this.items.push(item); }
  pop(): T | undefined { return this.items.pop(); }
  peek(): T | undefined { return this.items[this.items.length - 1]; }
  get size(): number { return this.items.length; }
}

const numStack = new Stack<number>();
numStack.push(1); numStack.push(2);
numStack.pop(); // 2 - typed as number

const strStack = new Stack<string>();
strStack.push('hello');
```

3.2 Type Guards & Narrowing

Narrowing is the process of refining a type within a conditional block. TypeScript tracks these refinements through control flow analysis.

```
// typeof guard - for primitives
function process(value: string | number) {
  if (typeof value === 'string') {
    return value.toUpperCase(); // narrowed to string
  }
  return value.toFixed(2); // narrowed to number
}

// instanceof guard - for class instances
function handleError(error: Error | string) {
  if (error instanceof Error) {
    return error.message; // narrowed to Error
  }
}
```

```
    return error;          // narrowed to string
  }

// in guard — check if property exists
interface Cat { meow(): void; }
interface Dog { bark(): void; }

function makeSound(animal: Cat | Dog) {
  if ('meow' in animal) {
    animal.meow(); // narrowed to Cat
  } else {
    animal.bark(); // narrowed to Dog
  }
}

// Custom type guard — user-defined narrowing function
function isString(value: unknown): value is string {
  return typeof value === 'string';
}

const val: unknown = 'hello';
if (isString(val)) {
  val.toUpperCase(); // OK — narrowed to string
}
```

3.3 Literal & Template Literal Types

```
// Literal types — restrict to exact values
type Direction = 'north' | 'south' | 'east' | 'west';
type DiceRoll  = 1 | 2 | 3 | 4 | 5 | 6;
type Bool      = true | false; // same as boolean

function move(dir: Direction) { /* ... */ }
move('north'); // OK
// move('up'); // Error — 'up' is not assignable to Direction

// Template literal types — build string types from patterns
type EventName = `on${string}`; // 'onClick', 'onSubmit', etc.

type Axis      = 'x' | 'y' | 'z';
type CSSProp   = `margin-${Axis}`; // 'margin-x' | 'margin-y' | 'margin-z'

type CRUD      = 'create' | 'read' | 'update' | 'delete';
type Permission = `${CRUD}:${string}`; // 'create:users', 'read:posts', etc.

// Practical: event listener types
type InputEvent = 'focus' | 'blur' | 'change';
type HandlerName = `on${Capitalize<InputEvent>}`; // 'onFocus' | 'onBlur' | 'onChange'
```

3.4 Mapped Types

Mapped types transform an existing type by iterating over its keys. They are how TypeScript's built-in utility types like `Partial` and `Readonly` are implemented.

```
// Syntax: { [K in keyof T]: ... }

// Make all properties optional
type Partial<T> = {
  [K in keyof T]?: T[K];
};

// Make all properties readonly
type Readonly<T> = {
  readonly [K in keyof T]: T[K];
};

// Make all properties nullable
type Nullable<T> = {
  [K in keyof T]: T[K] | null;
};

// Remap keys with 'as'
type Getters<T> = {
  [K in keyof T as `get${Capitalize<string & K>}`]: () => T[K];
};

interface User { name: string; age: number; }
type UserGetters = Getters<User>;
// { getName: () => string; getAge: () => number; }

// Remove keys matching a condition
type NonNullableProps<T> = {
  [K in keyof T as T[K] extends null | undefined ? never : K]: T[K]
};
```

3.5 Conditional Types

Conditional types express type logic as `if/else` decisions: `T extends U ? TypeIfTrue : TypeIfFalse`.

```
// Basic conditional type
type IsString<T> = T extends string ? 'yes' : 'no';

type A = IsString<string>; // 'yes'
type B = IsString<number>; // 'no'
```

```
// Distributive conditional types — apply to each member of a union
type ToArray<T> = T extends any ? T[] : never;
type C = ToArray<string | number>; // string[] | number[]

// Practical: extract only function properties
type FunctionKeys<T> = {
  [K in keyof T]: T[K] extends Function ? K : never;
}[keyof T];

interface MyObj {
  name: string;
  age: number;
  greet(): void;
  save(): Promise<void>;
}
type Methods = FunctionKeys<MyObj>; // 'greet' | 'save'

// NonNullable — remove null and undefined from a type
type NonNullable<T> = T extends null | undefined ? never : T;
type D = NonNullable<string | null | undefined>; // string
```

3.6 The infer Keyword

`infer` allows you to extract a type from within another type during a conditional type evaluation. It is what makes advanced utility types possible.

```
// Extract the return type of a function
type ReturnType<T> = T extends (...args: any[]) => infer R ? R : never;

function getUser() { return { name: 'Alice', age: 30 }; }
type UserType = ReturnType<typeof getUser>; // { name: string; age: number }

// Extract the parameter types of a function
type Parameters<T> = T extends (...args: infer P) => any ? P : never;

function add(a: number, b: number) { return a + b; }
type AddParams = Parameters<typeof add>; // [a: number, b: number]

// Unwrap a Promise
type Awaited<T> = T extends Promise<infer V> ? V : T;
type E = Awaited<Promise<string>>; // string
type F = Awaited<string>; // string (not a promise)

// Extract element type from array
type ElementType<T> = T extends (infer E)[] ? E : never;
type G = ElementType<string[]>; // string
type H = ElementType<number[]>; // number
```

3.7 Built-in Utility Types

TypeScript ships with a set of generic utility types that transform types in common ways. Learn these — you will use them constantly.

```
interface User {
  id:      number;
  name:    string;
  email:   string;
  password: string;
  age:     number;
}

// Partial<T> - all properties become optional
type UpdateUser = Partial<User>;
// { id?: number; name?: string; email?: string; ... }

// Required<T> - all optional properties become required
type FullUser = Required<User>;

// Readonly<T> - all properties become readonly
type ImmutableUser = Readonly<User>;

// Pick<T, K> - keep only specified keys
type PublicUser = Pick<User, 'id' | 'name' | 'email'>;
// { id: number; name: string; email: string }

// Omit<T, K> - remove specified keys
type SafeUser = Omit<User, 'password'>;
// { id: number; name: string; email: string; age: number }

// Record<K, V> - build an object type from keys and values
type Scores = Record<string, number>; // { [key: string]: number }
type RoleMap = Record<'admin' | 'user' | 'guest', string[]>;

// Exclude<T, U> - remove from union
type NoNull = Exclude<string | number | null, null>; // string | number

// Extract<T, U> - keep only matching union members
type Strings = Extract<string | number | boolean, string>; // string

// ReturnType<T> - get return type of a function
const getUser = () => ({ name: 'Alice', age: 30 });
type UserShape = ReturnType<typeof getUser>;

// Parameters<T> - get parameter types as a tuple
function add(a: number, b: number): number { return a + b; }
type AddArgs = Parameters<typeof add>; // [a: number, b: number]
```

PHASE 4

Object-Oriented TypeScript

4.1 Classes & Access Modifiers

TypeScript enhances JavaScript classes with access control keywords, typed properties, and constructor parameter shorthand.

```
class BankAccount {
  // Property declarations
  private balance: number;
  readonly id: string;
  public owner: string;

  constructor(owner: string, initialBalance: number) {
    this.owner = owner;
    this.balance = initialBalance;
    this.id = Math.random().toString(36).slice(2);
  }

  // Getter — access private state
  get currentBalance(): number {
    return this.balance;
  }

  deposit(amount: number): void {
    if (amount <= 0) throw new Error('Amount must be positive');
    this.balance += amount;
  }

  withdraw(amount: number): void {
    if (amount > this.balance) throw new Error('Insufficient funds');
    this.balance -= amount;
  }
}

const acc = new BankAccount('Alice', 1000);
acc.deposit(500);
console.log(acc.currentBalance); // 1500
// acc.balance = 9999; // Error — 'balance' is private
// acc.id = 'new-id'; // Error — 'id' is readonly
```

Constructor Shorthand

```
// Verbose version
class Point {
  public x: number;
  public y: number;
```

```
    constructor(x: number, y: number) {
        this.x = x;
        this.y = y;
    }
}

// Shorthand – access modifier in constructor parameter auto-creates the property
class Point {
    constructor(public x: number, public y: number) {}
}
// Identical result – x and y become public properties

class User {
    constructor(
        public readonly id: string,
        public name: string,
        private passwordHash: string,
    ) {}
}
```

4.2 Inheritance & Abstract Classes

```
// Abstract class – defines a blueprint, cannot be instantiated directly
abstract class Shape {
    abstract area(): number; // must be implemented by subclasses
    abstract perimeter(): number;

    // Concrete method – shared by all shapes
    describe(): string {
        return `Area: ${this.area().toFixed(2)}, Perimeter: ${this.perimeter().toFixed(2)}`;
    }
}

class Circle extends Shape {
    constructor(private radius: number) { super(); }

    area(): number { return Math.PI * this.radius ** 2; }
    perimeter(): number { return 2 * Math.PI * this.radius; }
}

class Rectangle extends Shape {
    constructor(private w: number, private h: number) { super(); }

    area(): number { return this.w * this.h; }
    perimeter(): number { return 2 * (this.w + this.h); }
}

// Polymorphism – treat all shapes uniformly
const shapes: Shape[] = [new Circle(5), new Rectangle(4, 6)];
shapes.forEach(s => console.log(s.describe()));

// new Shape(); // Error – cannot instantiate abstract class
```


4.3 Implementing Interfaces

```
// Interfaces as contracts for classes
interface Serializable {
  serialize(): string;
  deserialize(data: string): this;
}

interface Validatable {
  validate(): boolean;
  errors: string[];
}

// A class can implement multiple interfaces
class UserForm implements Serializable, Validatable {
  errors: string[] = [];

  constructor(
    public name: string,
    public email: string,
  ) {}

  validate(): boolean {
    this.errors = [];
    if (!this.name) this.errors.push('Name is required');
    if (!this.email.includes('@')) this.errors.push('Invalid email');
    return this.errors.length === 0;
  }

  serialize(): string {
    return JSON.stringify({ name: this.name, email: this.email });
  }

  deserialize(data: string): this {
    const parsed = JSON.parse(data);
    this.name = parsed.name;
    this.email = parsed.email;
    return this;
  }
}
```

PHASE 5

Advanced TypeScript

5.1 Discriminated Unions

Discriminated unions use a shared literal field (the 'discriminant') to let TypeScript know exactly which variant of a union you have in any given branch.

```
// Each variant has a unique literal 'type' field — the discriminant
type Success<T> = { status: 'success'; data: T; };
type Failure    = { status: 'error';   error: string; code: number; };
type Loading    = { status: 'loading'; };

type Result<T> = Success<T> | Failure | Loading;

function renderUser(result: Result<User>) {
  switch (result.status) {
    case 'loading': return 'Loading...';
    case 'error':   return `Error ${result.code}: ${result.error}`;
    case 'success': return `Hello, ${result.data.name}`;
    // TS knows exactly which variant is in each case
  }
}

// Exhaustiveness check with never
function assertNever(x: never): never {
  throw new Error('Unhandled case: ' + JSON.stringify(x));
}

function handle(result: Result<User>) {
  switch (result.status) {
    case 'loading': return renderLoading();
    case 'error':   return renderError(result);
    case 'success': return renderSuccess(result);
    default:        return assertNever(result); // compile error if case missed
  }
}
```

5.2 Recursive Types

Types can reference themselves. This allows you to model deeply nested or tree-structured data with full type safety.

```
// A tree node type that can have children of its own type
type TreeNode<T> = {
  value: T;
  children: TreeNode<T>[];
};

const tree: TreeNode<string> = {
  value: 'root',
  children: [
    { value: 'child1', children: [] },
    { value: 'child2', children: [
      { value: 'grandchild', children: [] }
    ]},
  ]
};

// Deep readonly - recursively make all nested properties readonly
type DeepReadonly<T> = {
  readonly [K in keyof T]: T[K] extends object ? DeepReadonly<T[K]> : T[K];
};

// JSON-compatible type
type JSONValue =
  | string | number | boolean | null
  | JSONValue[]
  | { [key: string]: JSONValue };

const config: JSONValue = {
  name: 'app',
  version: 1,
  features: { auth: true, darkMode: false },
  tags: ['web', 'mobile'],
};
```

5.3 Declaration Merging & Module Augmentation

```
// Declaration merging - add to an existing interface
// In your app code:
interface User { name: string; }
interface User { age: number; } // TS merges these
// User is now: { name: string; age: number }

// Module augmentation - extend a third-party library's types
// Example: add a custom property to Express's Request object

// In a .d.ts file or at the top of your module:
import 'express';
declare module 'express' {
  interface Request {
    user?: { id: string; role: string };
  }
}
```

```
// Now Express's Request has your custom 'user' property
app.get('/profile', (req, res) => {
  const userId = req.user?.id; // fully typed
});

// Augmenting global Window
declare global {
  interface Window {
    myAnalytics: { track(event: string): void };
  }
}
window.myAnalytics.track('page_view'); // no error
```

5.4 The satisfies Operator (TS 4.9+)

`satisfies` validates that a value matches a type without widening the inferred type — giving you both validation AND precise inference.

```
type Colors = 'red' | 'green' | 'blue';
type Palette = Record<Colors, string | [number, number, number]>;

// Problem with direct annotation: loses specifics
const palette1: Palette = {
  red: [255, 0, 0],
  green: '#00ff00',
  blue: [0, 0, 255],
};
// palette1.red - typed as string | [number, number, number]
// palette1.red.toUpperCase(); // Error - might be an array

// satisfies: validates the shape but keeps precise types
const palette2 = {
  red: [255, 0, 0],
  green: '#00ff00',
  blue: [0, 0, 255],
} satisfies Palette;

// Now TS knows the specific types of each key
palette2.green.toUpperCase(); // OK - inferred as string
palette2.red[0]; // OK - inferred as number[] tuple

// Also catches errors at the point of definition
// const bad = { red: true } satisfies Palette; // Error
```

5.5 tsconfig.json — Key Options

The `tsconfig.json` file controls how TypeScript compiles your code. Understanding the key options helps you write safer and more compatible TypeScript.

```
{
  "compilerOptions": {
    // Target JavaScript version for output
    "target": "ES2020",

    // Which built-in type definitions to include
    "lib": ["ES2020", "DOM"],

    // Enable all strict type-checking options
    "strict": true,
    // (strict enables all of the below):
    "noImplicitAny": true,      // error if type would be inferred as any
    "strictNullChecks": true,  // null/undefined are not assignable to other
types
    "strictFunctionTypes": true, // stricter function type checking

    // Additional safety
    "noUnusedLocals": true,    // error on unused variables
    "noUnusedParameters": true, // error on unused function parameters
    "noImplicitReturns": true,  // all code paths must return a value
    "exactOptionalPropertyTypes": true, // stricter optional property handling

    // Module settings
    "module": "NodeNext",
    "moduleResolution": "NodeNext",
    "esModuleInterop": true,

    // Output
    "outDir": "./dist",
    "rootDir": "./src",
    "declaration": true,    // emit .d.ts files
    "sourceMap": true
  },
  "include": ["src/**/*"],
  "exclude": ["node_modules", "dist"]
}
```

5.6 Writing Declaration Files (.d.ts)

Declaration files allow you to add TypeScript types to JavaScript libraries that don't include their own. They describe the shape of an existing module without any implementation.

```
// my-library.d.ts - type declarations for a plain JS library

// Describe a function
declare function greet(name: string): string;
```

```
// Describe a class
declare class EventEmitter {
  on(event: string, listener: (...args: any[]) => void): this;
  off(event: string, listener: (...args: any[]) => void): this;
  emit(event: string, ...args: any[]): boolean;
}

// Describe a module
declare module 'my-js-library' {
  export interface Config {
    apiKey: string;
    timeout: number;
  }

  export function init(config: Config): void;
  export function fetch<T>(url: string): Promise<T>;

  export default class Client {
    constructor(config: Config);
    get<T>(path: string): Promise<T>;
    post<T>(path: string, body: unknown): Promise<T>;
  }
}
```

PHASE 6

Ecosystem & Real-World Patterns

6.1 Zod — Runtime Validation

TypeScript types are erased at runtime. When data comes in from the outside world (APIs, user input, files), you can't rely on TS to validate it. Zod bridges the gap: it validates data at runtime AND infers TypeScript types from your schemas.

```
import { z } from 'zod';

// Define a schema
const UserSchema = z.object({
  id: z.number().int().positive(),
  name: z.string().min(1).max(100),
  email: z.string().email(),
  age: z.number().min(0).max(150).optional(),
  role: z.enum(['admin', 'user', 'guest']),
});

// Infer the TypeScript type from the schema
type User = z.infer<typeof UserSchema>;
// { id: number; name: string; email: string; age?: number; role:
'admin' | 'user' | 'guest' }

// Validate data at runtime
const rawData = await fetch('/api/user').then(r => r.json());

const result = UserSchema.safeParse(rawData);
if (result.success) {
  const user = result.data; // typed as User
  console.log(user.name.toUpperCase()); // fully safe
} else {
  console.error(result.error.issues); // detailed error messages
}

// Nested schemas
const AddressSchema = z.object({
  street: z.string(),
  city: z.string(),
});

const PersonSchema = z.object({
  name: z.string(),
  address: AddressSchema, // reuse schemas
  tags: z.array(z.string()), // arrays
});
```

6.2 Testing Typed Code with Vitest

```
// userService.ts
interface User { id: number; name: string; }

export async function getUser(id: number): Promise<User> {
  const res = await fetch(`/api/users/${id}`);
  if (!res.ok) throw new Error('User not found');
  return res.json();
}

// userService.test.ts
import { describe, it, expect, vi, beforeEach } from 'vitest';
import { getUser } from './userService';

describe('getUser', () => {
  beforeEach(() => { vi.resetAllMocks(); });

  it('returns a user when the response is OK', async () => {
    const mockUser = { id: 1, name: 'Alice' };

    // Mock fetch globally
    global.fetch = vi.fn().mockResolvedValue({
      ok: true,
      json: () => Promise.resolve(mockUser),
    } as Response);

    const user = await getUser(1);
    expect(user).toEqual(mockUser);
    expect(user.name).toBe('Alice');
  });

  it('throws when the response is not OK', async () => {
    global.fetch = vi.fn().mockResolvedValue({ ok: false } as Response);

    await expect(getUser(99)).rejects.toThrow('User not found');
  });
});
```

6.3 Patterns — Putting It All Together

Pattern: The Repository Pattern (Generic + OOP)

```
// A generic, fully typed data access layer

interface Entity { id: number; }

interface Repository<T extends Entity> {
  findById(id: number): Promise<T | null>;
  findAll(): Promise<T[]>;
  create(data: Omit<T, 'id'>): Promise<T>;
}
```



```
    update(id: number, data: Partial<T>): Promise<T | null>;
    delete(id: number): Promise<boolean>;
  }

class InMemoryRepository<T extends Entity> implements Repository<T> {
  private store = new Map<number, T>();
  private nextId = 1;

  async findById(id: number): Promise<T | null> {
    return this.store.get(id) ?? null;
  }

  async findAll(): Promise<T[]> {
    return Array.from(this.store.values());
  }

  async create(data: Omit<T, 'id'>): Promise<T> {
    const entity = { ...data, id: this.nextId++ } as T;
    this.store.set(entity.id, entity);
    return entity;
  }

  async update(id: number, data: Partial<T>): Promise<T | null> {
    const existing = this.store.get(id);
    if (!existing) return null;
    const updated = { ...existing, ...data };
    this.store.set(id, updated);
    return updated;
  }

  async delete(id: number): Promise<boolean> {
    return this.store.delete(id);
  }
}

// Use it for any entity
interface Product extends Entity { name: string; price: number; }
const products = new InMemoryRepository<Product>();

await products.create({ name: 'Widget', price: 9.99 });
const all = await products.findAll(); // Product[]
```

Pattern: Type-Safe Event Bus

```
// Define your event map - event name to payload type
interface AppEvents {
  'user:login': { userId: string; timestamp: Date; };
  'user:logout': { userId: string; };
  'cart:updated': { itemCount: number; total: number; };
  'error': { message: string; code: number; };
}

class TypedEventBus<Events extends Record<string, unknown>> {
  private listeners = new Map<keyof Events, Set<Function>>();

  on<K extends keyof Events>(
```

```
    event: K,
    listener: (payload: Events[K]) => void
  ): () => void {
    if (!this.listeners.has(event)) this.listeners.set(event, new Set());
    this.listeners.get(event)!.add(listener);
    return () => this.listeners.get(event)?.delete(listener);
  }

  emit<K extends keyof Events>(event: K, payload: Events[K]): void {
    this.listeners.get(event)?.forEach(fn => fn(payload));
  }
}

const bus = new TypedEventBus<AppEvents>();

// Fully typed – autocomplete on event names and payloads
bus.on('user:login', ({ userId, timestamp }) => {
  console.log(`User ${userId} logged in at ${timestamp}`);
});

bus.emit('user:login', { userId: 'ul23', timestamp: new Date() });
// bus.emit('user:login', { userId: 123 }); // Error – userId must be string
```

You've reached the end of the coursework.

Now close the document and write code. The only way to learn TypeScript is by building things.